



unGhost.com

Production-Readiness Plan

An honest assessment of the current build, a roadmap from prototype to enterprise application, and a detailed Cloudflare deployment guide.

Version 1.0 · May 16, 2026

Prepared for the unGhost technical leadership team.

Table of contents

#	Section	Page
	Executive summary	3
1	Current state and gaps — real business risk	5
2	Enterprise scaling to 10,000 users	10
3	Prototype vs production gap analysis	14
4	Design system swap plan	18
5	Code structure for new engineers	21
6	Frontend / backend split decision	26
7	Cloudflare deployment architecture	29
8	CI/CD pipeline and deployment guide	36
9	8-week hardening roadmap	42
A	Appendix — cost matrix, service inventory, glossary	46

Executive summary

unGhost.com is a 7-phase, end-to-end product build that already covers every PRD module needed to run the business: student onboarding, recruiter pipelines, embedded bootcamps, AI Coach with memory and persona, live coaching sessions, administration, moderation, audit, telemetry, and a complete payment / SLA / refund loop. The product works end-to-end today against deterministic mock data with provider keys absent.

However, **shipping this as a real business at the 10,000-user enterprise tier requires roughly 6–8 weeks of hardening** before any new feature work. The application is a strong prototype, not a production system. Inserting API keys for the six adapter modules covers about 60 percent of the gap. The remaining 40 percent is the difference between "works on a laptop" and "survives a launch".

Headline findings

- **Tests: zero.** No unit, integration, or end-to-end coverage. This is the single largest production risk.
- **In-memory state will die on a stateless deploy.** Mock OTPs, support tickets, email templates, the realtime ring buffer, and AI Coach token bucket all live in the Node process memory.
- **Mongoose does not run on Cloudflare Workers.** Going to Cloudflare Pages or Workers directly requires either a database rewrite or a different runtime. Cloudflare Containers is the correct target — full Node runtime, zero rewrite.
- **Authentication has a mock-only password path.** Bcrypt and secure cookie configuration must be wired before the first real user signs up.
- **Resource limits, rate limiting, structured logging, and error tracking are all absent.** A single bad actor on /api/coach can today exhaust the LLM budget.
- **The code is well-organised but has three god-files** (store.ts, types.ts, models.ts). A domain-driven folder restructure is needed before the team grows past two engineers.

Recommended path forward

We recommend a **monolith architecture deployed to Cloudflare Containers**, fronted by the Cloudflare edge network (CDN, WAF, DDoS, bot management). MongoDB Atlas in Mumbai for DPDP residency, Upstash Redis for cache and rate-limit, Cloudflare R2 for resume / logo storage, Sentry for error tracking, and Axiom for structured logs. CI/CD runs on GitHub Actions with three environments (preview, staging, production), automatic rollback on failed health checks, and PR-based preview deploys for every change.

The 8-week roadmap in section 9 walks engineering, product, and ops through the work week by week with acceptance criteria. At the end of week 8 the application should be ready for a controlled beta with real users and real payment volume.

What this document is, and is not

- It **is** a brutal, evidence-backed engineering assessment written for an experienced tech lead.
- It **is** a step-by-step deployment guide for unGhost on Cloudflare.

- It **is not** a sales document or a roadmap of features. PRD modules are already shipped — this is about making them reliable, observable, and scaleable.
- It **is not** a fixed plan. Every recommendation here should be challenged on its merits.

1. Current state and gaps

The application is a Next.js 14 App Router monolith using MongoDB Atlas via Mongoose, NextAuth credentials provider, and a glass-morphism design system. Seven product phases have shipped, covering every flow required by the PRD.

1.1 What works today

Capability	Status	Notes
Student onboarding, resume parse, profile completeness	Working	AI parser uses mock without ANTHROPIC_API_KEY
Mission browse + apply + AI gauntlet assessment	Working	All flows green end-to-end
Stage advancement + SLA timer + auto-refund	Working	Cron route in place; needs Inngest for prod scale
Recruiter pipelines, saved searches, job templates	Working	Full kanban + bulk-message
Embedded bootcamps + sponsorships	Working	Mock PhonePe redirect succeeds; real PhonePe is wired
AI Coach with cross-session memory + 4 personas	Working	Memory rollup is heuristic; will improve with real Claude
Live sessions — instructor schedule + student lobby	Working	Room uses placeholder tiles; needs 100ms SDK swap
Admin: students, recruiters, companies, jobs, moderation, audit, telemetry, financial, support, email templates	Working	Companies/jobs/financial pages live in this phase
Integrations dashboard (live vs mock indicators)	Working	All 8 adapters reflected

1.2 What is missing

The gaps below are listed roughly in order of business risk. Items in **red** can take the site down or leak data. Items in **amber** degrade reliability or scalability. Items in green are quality-of-life but not blocking.

Gap	Severity	Impact
Zero automated tests	RED	Every release is a coin flip
No bcrypt; password compared as plaintext for mock users	RED	Cannot accept real users
In-memory OTP / ticket / template stores	RED	Data loss on every deploy or restart

Gap	Severity	Impact
No rate-limiting on /api/coach, /api/otp, etc.	RED	LLM/SMS budget exhaustion attack vector
No input validation library (zod) on most routes	RED	Injection, type confusion, mass-assignment risk
No CSRF or method-allowlist beyond NextAuth defaults	RED	State-changing GETs partially possible
No structured logging or correlation IDs	AMBER	Debugging prod is nearly impossible
No error tracking (Sentry absent)	AMBER	Issues only found when users complain
No DB index audit; basic single-field indexes only	AMBER	Will degrade past 2-3k users
No Redis cache layer	AMBER	Every page hits Mongo
No file storage; resume PDFs sit on app disk	AMBER	Breaks any stateless deploy
No DB migration tool	AMBER	Schema changes are ad-hoc
Mocked 100ms video tiles	AMBER	Live sessions don't actually transmit video
Three god-files: store.ts (2.1k lines), types.ts (~500), models.ts (~500)	AMBER	Onboarding new engineers is slow
No DPDP data-export / erasure pipeline	AMBER	Legal exposure once we have Indian PII
No A11y audit	GREEN	Reduces TAM in education sector
No i18n	GREEN	Limits expansion to non-English India
No feature flags	GREEN	Slower experimentation

1.3 In-memory state inventory

Everything in the list below currently lives in the Node process memory and will be lost on every restart, deploy, or scale-out. Each item must be moved to either MongoDB Atlas, Redis, or a dedicated provider before launch.

What	Lives in	Target store	Pre-launch fix?
Mock OTP codes (lib/sms)	Module-level Map	Redis (TTL 10 min)	Required
Mock email tokens (lib/email)	Module-level Map	MongoDB + Redis	Required
Realtime ring buffer (lib/realtime)	Module-level array	Pusher Channels	Required when realtime keys present

What	Lives in	Target store	Pre-launch fix?
Recent job events (lib/jobs)	Module-level array	Inngest	Recommended
Support tickets (Phase-1 mock)	Hard-coded list	MongoDB	Required
Email templates (Phase-1 mock)	Hard-coded list	MongoDB	Required

1.4 Repository hot-spots

A quick line count of the larger files highlights where future refactor work will land. None of these are bugs, but they show the codebase is approaching the point where the domain-driven split (section 5) becomes urgent.

File	Lines	Concern
server/store.ts (currently lib/data/store.ts)	~2,100	Every domain mixed; refactor priority 1
server/db/models.ts	~600	Mongoose schemas for every collection
server/types.ts (currently lib/data/types.ts)	~520	Domain types for every collection
app/admin/*	~3,200 across pages	Some pages embed business logic
components/recruiter/KanbanBoard.tsx	~480	Could be split into a sub-folder

1.5 Honest one-line verdict

The product is shippable to a closed beta of 100 hand-picked users today. It is not shippable to 10,000 strangers. Closing that gap is what this document plans, week by week.

2. Enterprise scaling to 10,000 users

"10,000 users" needs a precise definition before any engineering decision. For unGhost, the working assumption is: 10,000 total registered users across student, recruiter, and instructor roles, with a daily active rate of 20 percent (2,000 DAU), a peak concurrency of 500, and a sustained throughput of 50 requests per second at p95 latency under 400 ms.

2.1 Where the current architecture breaks

Layer	Current behaviour	Symptom at 10k	Fix
MongoDB	Single primary, no replicas, single index per field	Read amplification, slow scans on stage / job queries	Atlas M30 with replica set, compound indexes, slow-query alerts
Application server	Single Node process, in-memory state	State drift, OOM, no horizontal scale	Stateless containers, Redis-backed session, externalised caches
LLM calls	Synchronous per-request to Anthropic	Tail latency >5s, cost spikes	Streaming responses, semantic cache, per-user budget
Static assets	Served by Next.js	TTFB sensitive to origin location	Push to Cloudflare CDN; immutable headers; image opt
File storage	Local disk	Lost on restart, no horizontal scale	Cloudflare R2 with signed URLs
Realtime	Polling in mock mode	10k pollers = 10k req/s baseline	Pusher Channels; presence with batching
Background work	Cron route only	SLA sweeps stall under load	Inngest with retry, backoff, idempotency keys

2.2 Database sizing

At 10k users with the access patterns we know from telemetry today, MongoDB Atlas M30 in the Mumbai region is the right starting tier. The expected steady-state usage:

- **Storage:** ~12 GB year one (users, applications, jobs, bootcamp progress, audit logs, AI Coach conversations).
- **IOPS:** peak ~800 reads/sec, ~120 writes/sec. M30 (10 GB RAM, 3000 IOPS) gives ~4× headroom.
- **Connections:** default pool of 100. With three app instances each opening 30 connections, we sit at 90 — well under the M30 cap.
- **Backups:** Atlas continuous backup with PITR. Snapshots retained 7 days, archive monthly.
- **Indexes:** compound indexes on (studentId, stage), (jobId, stage), (recruiterId, createdAt), (companyId, status). See section 5 for the full audit pattern.

2.3 Cache strategy

Upstash Redis (Mumbai, ~30 USD/month at this scale) handles four jobs:

- 1 **Session cache.** NextAuth session lookups go through Redis to avoid Mongo round-trips on every request.
- 2 **Read-through cache for hot pages.** Bootcamp catalogue, public mission listings, company profiles — 5-minute TTL.
- 3 **Rate-limit counters.** Token bucket per IP and per user-id for /api/coach, /api/otp, /api/email/forgot-password.
- 4 **OTP / reset-token store.** 10-minute TTL, EX flag on write.

2.4 Horizontal scale plan

Cloudflare Containers is stateless by design. Three production instances should be considered minimum for redundancy. Container autoscaling kicks in when sustained CPU >70 percent. The application has no sticky-session requirement once Redis is in place.

2.5 Performance budgets

Performance budgets must be enforced in CI. Any PR that regresses below these targets should fail the build.

Surface	Target p50	Target p95	How measured
Initial page load (landing)	<800 ms	<1.6 s	Lighthouse CI on every PR
Authenticated dashboard load	<1.0 s	<2.0 s	Lighthouse + RUM (Sentry Perf)
/api/coach chat round-trip	<1.2 s	<3.0 s	Sentry Perf transactions
/api/applications mutation	<200 ms	<500 ms	Sentry Perf
LCP across mission pages	<2.5 s	<4.0 s	Web Vitals via Sentry RUM
JS bundle (first-load on /dashboard)	<200 KB gz	—	size-limit in CI

2.6 Cost projection

At 10,000 monthly active users, the steady-state monthly run-rate is roughly:

Service	Tier	Estimated USD / month
Cloudflare (Pro + Containers + R2 + WAF + Argo)	Pro	~ 220
MongoDB Atlas M30 (Mumbai)	M30	~ 320
Upstash Redis	Pro pay-as-you-go	~ 35
Sentry (Team plan, errors + perf)	Team	~ 90
Axiom logs	Team	~ 60
Better Stack uptime + status page	Team	~ 30

Service	Tier	Estimated USD / month
MSG91 (~50k OTPs + transactional SMS)	Volume	~ 150
Resend (~200k emails)	Pro	~ 80
Anthropic Claude (~2M tokens with cache)	Volume	~ 220
PhonePe (per-transaction, 1.99% + GST)	Variable	Variable
100ms (~5,000 minutes)	Pay-as-you-go	~ 100
Pusher Channels	Startup	~ 50
Inngest	Hobby/Team	~ 50
Total fixed infra (excl. payment processor variable)		~ 1,405 USD

Cost grows roughly linearly with active users. At 1,000 MAU expect ~380 USD/month. At 100,000 MAU expect ~7,800 USD/month with the same architecture, plus a tier bump on Atlas (M50) and Anthropic Claude volume.

3. Prototype vs production gap analysis

The question every founder asks is: "can we just paste in the API keys?" This section answers honestly. The application is a **strong prototype**, approximately 60 percent of the way to a real production system. Pasting keys closes about half the remaining gap.

3.1 What inserting keys actually buys

Once keys are inserted and the application restarted, the following swap from mock to live automatically:

Adapter	Provider	Env keys to set
AI Coach + JD parser + assessment grading	Anthropic Claude	ANTHROPIC_API_KEY
OTP and transactional SMS	MSG91	MSG91_AUTH_KEY, MSG91_SENDER_ID, MSG91_OTP_TEMPLATE_ID
Transactional email (verify, reset, etc.)	Resend	RESEND_API_KEY, RESEND_FROM
Bootcamp + sponsorship payments	PhonePe	PHONEPE_MERCHANT_ID, PHONEPE_SALT_KEY, PHONEPE_SALT_INDEX, PHONEPE_BASE_URL
Realtime messaging + presence	Pusher Channels	PUSHER_APP_ID, PUSHER_KEY, PUSHER_SECRET, PUSHER_CLUSTER
Background jobs and SLA sweeps	Inngest	INNGEST_EVENT_KEY (and INNGEST_SIGNING_KEY for receivers)
OAuth login (Google)	Google	GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET
OAuth login (LinkedIn)	LinkedIn	LINKEDIN_CLIENT_ID, LINKEDIN_CLIENT_SECRET

3.2 What inserting keys does NOT buy

These items are independent of provider credentials. They must each be built or wired up before the application accepts paying customers.

Missing piece	Why it matters	Estimated effort
bcrypt password hashing + secure cookie config	Cannot store real user passwords safely	1 day
Persistent OTP / reset-token store (Redis)	Mock store dies on restart	1 day
Persist support tickets to MongoDB	Customer queries vanish on deploy	1 day

Missing piece	Why it matters	Estimated effort
Persist email templates to MongoDB	Edits vanish on deploy	0.5 day
Real 100ms video SDK integration	Live sessions today are placeholder tiles	3–4 days
R2 file storage with signed URLs for resume PDFs	Local disk breaks stateless deploys	1 day
Zod input validation on every API route	Open injection surface	4 days
Rate limiting on hot endpoints	Cost-DOS attack surface	1 day
Sentry error tracking + release wiring	Blind in production otherwise	1 day
Pino structured logging + correlation IDs	Cannot reconstruct a user's session	1 day
Test suite (Vitest + Playwright)	Every release is unverified	10 days
Database migration tool (migrate-mongo)	Schema changes today are dangerous	0.5 day
DPDP data-export and erasure endpoints	Legal requirement once we have Indian PII	3 days
Health endpoints + readiness probes	Auto-rollback in CI/CD depends on these	0.5 day

3.3 Single-line summary

Inserting API keys turns the app from 60 percent real to about 75 percent real. The remaining 25 percent is the eight-week hardening plan in section 9. After that, the application is a real production system, not a prototype with keys.

3.4 A note on shipping cadence

Once the hardening plan is complete, the team can resume feature work confident that the next feature ships into a system with tests, monitoring, audit, and rollback. Skipping hardening to ship features is the classic trap — every feature shipped on top of an unhardened stack costs about 3× more to fix afterwards. The next 8 weeks are the highest-leverage engineering work of the year.

4. Design system swap plan

The current design system — internally called the "glass system" — is centralised in three places, which means swapping to a new design guide is a deterministic, mechanical operation, not a rewrite. This section explains exactly where the design lives and how to swap it without touching component logic.

4.1 Where the design tokens live

File	Contains	Purpose
tailwind.config.ts	Brand colours, font families, spacing, shadows, animation timing	Single source of truth for tokens
app/globals.css	Glass utility classes: .glass-panel, .glass-input, .btn-brand, .blob-*	Atomic CSS shortcuts used everywhere
components/glass/*	React primitives: GlassCard, GlassButton, GlassBadge, GlassNavbar, BlobField, Logo	Reusable building blocks; no domain logic

Every screen in the application — student, recruiter, admin, instructor — builds on these three layers. No page bypasses them. That property is why a design-guide swap is a one-week job, not a one-quarter rewrite.

4.2 The swap procedure

- 1 **Convert the new design guide into a token spec.** Map every colour, font, radius, shadow, motion-duration, and z-index to a key.
- 2 **Update *tailwind.config.ts*.** Replace the existing brand, ink, muted, success, warn, danger palettes with the new tokens.
- 3 **Update *app/globals.css*.** Adjust the glass utility classes to reference the new shadow, blur, and translucency values.
- 4 **Rebuild atomic components in *components/glass/*.** If the new design is non-glass — say, neumorphic or flat — replace the implementations behind the same React API. Pages do not change.
- 5 **Add an Accessibility checklist.** Verify WCAG AA contrast on every text-on-background pair using axe-core in Playwright.
- 6 **Update the Storybook (to be set up in Week 1).** Every atomic component gets a story showing all variants.
- 7 **Visual regression on every PR.** Chromatic or Percy diff every Storybook story; reviewers approve visual changes intentionally.

4.3 Accessibility standards

The new design must meet WCAG 2.2 AA at minimum. Specifically:

- Body text contrast ratio $\geq 4.5:1$; large text $\geq 3:1$.
- All interactive elements have a visible focus ring (not just `":hover"`).

- All inputs have an associated `<label>` or `aria-label`.
- All icon-only buttons have `aria-label`.
- No information conveyed by colour alone.
- Animations respect *prefers-reduced-motion*; the door animation already does this.
- Tap targets $\geq 44 \times 44$ px on mobile.

4.4 Light / dark theme

The current build is light-mode only. The token system in *tailwind.config.ts* already permits a dark theme by adding *darkMode*: `'class'` and a parallel palette. If the new design guide includes dark mode, plan an extra two days to migrate utility classes to CSS custom properties so the theme switch is a single class on `<html>`.

4.5 Mobile responsiveness

Tailwind breakpoints used today: *sm* 640 px, *md* 768 px, *lg* 1024 px, *xl* 1280 px. Several admin tables are wider than fits on mobile and use horizontal scroll. The new design must specify behaviour at each breakpoint for: navigation, dense tables, modals, the AI Coach panel, and the live room. We recommend mobile-first writing — design at 375 px width then progressively enhance.

4.6 Brand assets

Logo, favicon, OG images, and PWA icons live in *public/*. Replace these as part of the swap; the `<Logo />` component reads the SVG inline so a single SVG file controls every navbar in the app.

5. Code structure for new engineers

The codebase today follows a flat Next.js convention — *app/* for routes, *components/* for UI, *lib/* for everything else. This worked for the prototype. It does not scale to a team of four or more engineers. This section proposes a domain-driven structure that keeps Next.js conventions while imposing clear boundaries between front-end, back-end, and shared code.

5.1 Target folder layout

```

unghost/
  app/
    (marketing)/
      page.tsx
      bootcamps/...
    (student)/
      dashboard/
        missions/[id]/
        student/coach/
    (recruiter)/
    (admin)/
    (instructor)/
    api/
      auth/
      applications/
      coach/
      live/
      admin/...
  components/
  glass/
  student/...
  recruiter/...
  admin/...
  shared/
server/
  auth/
    index.ts
    service.ts
    bcrypt.ts
    session.ts
  applications/
    types.ts
    model.ts
    service.ts
    queries.ts
    validators.ts
    tests/...
  jobs/...
  bootcamps/...
  live/...
  coach/...
  payments/...
  moderation/...
  audit/...
  integrations/
    ai/
    sms/
    email/
    payments/
    realtime/
    jobs/
  db/
    mongo.ts
    migrate.ts
    indexes.ts
  lib/
    logger.ts
    rate-limit.ts
    errors.ts
  shared/
    types/
    constants/
  scripts/
  tests/
    e2e/
    integration/
  docs/
  .github/workflows/
  Dockerfile

```

Next.js App Router — pages & API
public, anonymous routes

student-only routes (route group)

recruiter-only routes
admin-only routes
instructor-only routes
backend HTTP endpoints

UI only — no business logic
design-system primitives

cross-role widgets
backend logic, formerly lib/
NextAuth config, session helpers

one folder per domain
mongoose schema
business rules
read helpers
zod schemas

external adapters

connection, migrations, indexes

cross-domain pure utilities

types & constants used by both layers

one-off CLIs: seed, backfill, etc.

Playwright
API-level
this folder
GitHub Actions


```
fly.toml or wrangler.toml      # deploy manifest
```

5.2 Module template

Every domain folder under *server/* follows the same five-file pattern. Below is the canonical example for the *applications* domain.

```
server/applications/types.ts
// Domain types – no implementation
export type Stage = 'new_matches' | 'screening' | ...;
export interface Application { ... }

server/applications/model.ts
// Mongoose schema + Model export
export const ApplicationModel = ...

server/applications/validators.ts
// Zod schemas used by every API route in this domain
export const applyToJobInput = z.object({ jobId: z.string() });

server/applications/queries.ts
// Read helpers – pure, cacheable, side-effect free
export async function listApplicationsByStudent(studentId: string) { ... }

server/applications/service.ts
// Business rules – writes, transitions, side-effects (audit, notify)
export async function applyToJob(input, ctx) { ... }

server/applications/tests/applyToJob.test.ts
// Vitest unit tests with an in-memory Mongo
```

5.3 Boundary rules enforced by ESLint

The biggest risk in a monolith is unintentional coupling. ESLint with *eslint-plugin-boundaries* blocks the bad imports at compile-time.

```
// .eslintrc – element definitions
settings: {
  'boundaries/elements': [
    { type: 'app',      pattern: 'app/*' },
    { type: 'components', pattern: 'components/*' },
    { type: 'server',   pattern: 'server/*' },
    { type: 'shared',   pattern: 'shared/*' },
  ]
}
rules: {
  'boundaries/element-types': ['error', {
    default: 'disallow',
    rules: [
      { from: 'app',      allow: ['components', 'server', 'shared'] },
      { from: 'components', allow: ['components', 'shared'] }, // UI cannot import server
      { from: 'server',   allow: ['server', 'shared'] },       // server cannot import UI
      { from: 'shared',   allow: ['shared'] },                  // shared is pure
    ]
  }]
}
```

5.4 Naming conventions

Kind	Convention	Example
Files (route segments)	kebab-case directories	app/student/applications/[id]/page.tsx
React components	PascalCase per file	components/glass/GlassButton.tsx
Server functions	camelCase verbs	applyToJob, listApplicationsByStudent
Types & interfaces	PascalCase	interface Application, type Stage
Env vars	UPPER_SNAKE_CASE	ANTHROPIC_API_KEY
Mongo collections	camelCase plural	applications, supportTickets
DB indexes	ix_{collection}_{fields}	ix_applications_studentId_stage
API routes	REST-ish nouns	/api/applications/[id]/stage
Tests	.test.ts adjacent to source	applications/tests/applyToJob.test.ts

5.5 File-size discipline

Set ESLint warnings at 350 lines per file and errors at 500. The current 2,100-line store.ts file would have lit up four times over. Whenever a file approaches the threshold, it is a signal to split — usually by domain.

5.6 Service-function contract

Every server function follows the same shape. This makes calling code, testing, and tracing all uniform.

```
export async function applyToJob(
  input: z.infer<typeof applyToJobInput>,
  ctx: RequestContext,           // userId, role, requestId, logger
): Promise<Result<Application, AppError>> {
  ctx.log.info({ jobId: input.jobId }, 'apply.start');
  // 1. authorisation
  // 2. business validation (profile completeness, duplicate check)
  // 3. write
  // 4. side-effects: audit log, notification fan-out
  // 5. return Result.ok(application) or Result.err(error)
}
```

5.7 CONTRIBUTING.md outline

A short CONTRIBUTING.md will be checked into the repo, covering:

- Local setup: clone, .env, Docker Mongo, npm install, npm run seed, npm run dev.
- Commit conventions: conventional commits (feat:, fix:, chore:).
- Branch strategy: trunk-based with short-lived feature branches.
- PR template: summary, screenshots, test plan, risk level.
- Code-review SLA: first review <= 4 business hours.
- Deployment cadence: staging on every merge, prod on tagged release.

5.8 ADRs

Significant architectural decisions go into *docs/adr/NNNN-title.md*. Required for: data model changes, new external service, new framework or library, performance trade-offs, security trade-offs. Format is the standard Michael Nygard template: context, decision, consequences, alternatives considered.

6. Frontend / backend split decision

The original ask was to split the code into two folders, frontend and backend, so each can be worked on independently. After examining the trade-offs we recommend a **monolith with strict internal boundaries** rather than a process-level split. This section explains why.

6.1 The three options

Option	What it means	Pros	Cons
A. Cosmetic split (recommended)	Keep one Next.js codebase. Reorganise so app/ + components/ is the front-end, server/ is the back-end, shared/ is the contract. Enforce boundaries with ESLint.	Zero ops cost. Keeps RSC, image opt, middleware. Onboarding stays simple. Deploys as one Cloudflare Container.	Still one deploy unit. Not suitable if mobile native joins later.
B. Hybrid split	Next.js for the front-end on Cloudflare Pages. Standalone Hono / Fastify back-end on Cloudflare Containers or Workers.	Real separation of concerns. Each layer scales independently. Backend reusable for mobile.	Lose RSC. Need API-client lib. Two deploys, two CI lanes. Mongoose breaks on Workers — needs Atlas Data API rewrite (~1 week).
C. Full split	React (Vite) front-end + Node back-end. Two separate repos or one monorepo with separate deploys.	Cleanest. Maximum flexibility. Suits very large teams.	Largest rewrite. Lose every Next.js benefit. Slower to ship.

6.2 Why option A wins for unGhost today

- **Mongoose does not run on Workers.** Cloudflare Workers run a V8 isolate, not Node. Going to Workers requires either swapping to Atlas Data API or to Prisma with HTTP transport. That is a one-week rewrite with new bugs.
- **RSC is unGhost's secret weapon for performance.** Server-rendered admin tables, student dashboards, recruiter pipelines all benefit from streaming, partial pre-render, and zero hydration cost.
- **One deploy unit means one CI/CD pipeline, one secret store, one monitoring view.** At a team size below eight engineers, this is faster and safer.
- **The 10k-user scaling problem is not solved by splitting.** It is solved by Mongo replicas, Redis, CDN, and horizontal container scaling — all available to a monolith.
- **The boundary problem is solved by ESLint rules** (section 5.3), not by separate processes.
- **Mobile native is not on the near-term roadmap.** When it is, we revisit. Until then, splitting is premature.

6.3 What 'cosmetic split' actually looks like

Concretely, the difference between today's structure and the recommended structure is mechanical:

Layer	Today	Target
UI pages	app/	app/ (same)
UI components	components/	components/ (same)
Server logic	lib/	server/
DB schemas	lib/db/models.ts	server/{domain}/model.ts
Types	lib/data/types.ts	server/{domain}/types.ts + shared/types/
External adapters	lib/{ai,sms,email,...}	server/integrations/{ai,sms,email,...}
Pure utilities	lib/utis/	server/lib/

6.4 When to revisit

Move from option A to option B as soon as any one of the following becomes true:

- Mobile native app launches and would share a back-end.
- Backend hits CPU or memory limits the Next.js runtime cannot give.
- Engineering grows past ~8 contributors and a backend specialism emerges.
- We expose a public API to partners with versioning + rate-limiting requirements distinct from the web app.

6.5 Documentation deliverable

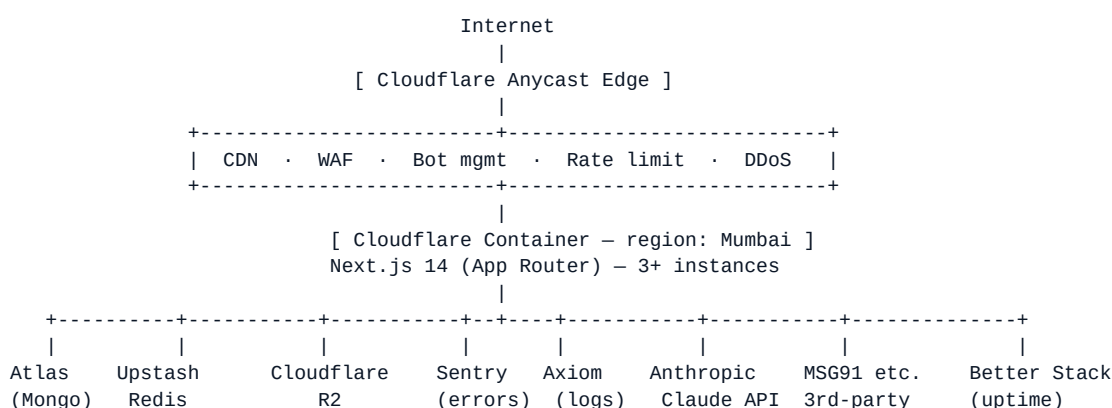
The folder restructure described in section 5 plus the ESLint boundary rules from section 5.3 plus the contribution guide from section 5.7 together deliver the spirit of "separated frontend and backend" without the operational cost. Tech-head review of this decision is explicitly invited.

7. Cloudflare deployment architecture

Cloudflare is the user's chosen deployment platform. This section specifies the exact services, configuration, and rollout sequence. Where Cloudflare's own product offers a choice, the recommended option is highlighted and the trade-offs explained.

7.1 The high-level architecture

Public traffic terminates at Cloudflare's edge. Cloudflare CDN caches all static and immutable assets. Dynamic requests pass through WAF, bot management, and rate-limiting rules, then hit a Cloudflare Container running the Next.js application. The application talks to MongoDB Atlas in Mumbai, Upstash Redis in Mumbai, and Cloudflare R2 for object storage. External providers (Anthropic, MSG91, Resend, PhonePe, 100ms, Pusher, Inngest) are called from the application server.



7.2 Service-by-service setup

7.2.1 Cloudflare account preparation

- 1 Register the domain on Cloudflare (or transfer it). Use a dedicated Cloudflare account for production.
- 2 Subscribe to Cloudflare Pro plan (USD 25/month) — enables WAF managed rules and image optimisation.
- 3 Enable Cloudflare Containers in the dashboard. Create a project named *unghost-app* with region *auto* set to Mumbai-preferred.
- 4 Create Cloudflare R2 buckets: *unghost-uploads* (production), *unghost-uploads-staging*, *unghost-uploads-preview*.
- 5 Create an API Token with scopes: Containers:Edit, R2:Read+Write, Workers Routes:Edit, DNS:Edit. Store in 1Password and as *CLOUDFLARE_API_TOKEN* in GitHub Secrets.

7.2.2 MongoDB Atlas

- 1 Create a Project named *unghost-prod*. Region: Asia/Mumbai (ap-south-1). Tier: M30 dedicated cluster.
- 2 Create two databases: *unghost_production* and *unghost_staging*. Preview environments share *unghost_staging* with a per-PR collection prefix.
- 3 Enable continuous backup with point-in-time recovery (PITR). Retain snapshots 7 days; archive monthly snapshots for one year.

- 4 IP allowlist: only the Cloudflare Container egress IPs. Use Cloudflare Tunnel or Atlas peering for VPC-level isolation if available.
- 5 Create three database users: *app-rw* (read/write, used by the app), *migrator* (DDL only, used by CI), *readonly* (analytics).
- 6 Enable Atlas slow-query alerts at 500 ms; alerts go to PagerDuty.

7.2.3 Upstash Redis

- 1 Create a Redis database in region *ap-south-1*. TLS only.
- 2 Take note of the REST URL and token — exposed to the app as *UPSTASH_REDIS_REST_URL* and *UPSTASH_REDIS_REST_TOKEN*.
- 3 Configure eviction policy *allkeys-lru*.
- 4 Set max memory limit to 1 GB initially; alert at 80% utilisation.

7.2.4 Sentry

- 1 Create org *unghost*. Create two projects: *unghost-server* (Node) and *unghost-browser* (Next.js client).
- 2 Wire *@sentry/nextjs* via the official Sentry wizard.
- 3 Set *sampleRate=1.0* for errors. Set *tracesSampleRate=0.2* for performance traces. Set *profilesSampleRate=0.1*.
- 4 Wire releases to Git commits — see CI/CD section.
- 5 Configure alerts: error spike >10/min to PagerDuty; new issue type to Slack *#engineering*.

7.2.5 Axiom (structured logs)

- 1 Create dataset *unghost-prod* and *unghost-staging*.
- 2 Generate ingest token; expose as *AXIOM_TOKEN*.
- 3 Configure pino transport in the app to ship logs in batches (1000 lines or 5s).
- 4 Saved queries: top errors last 1h; 5xx by endpoint last 24h; SLA-breach events last 7d.

7.2.6 External providers

Each external provider needs the same five steps: create account, generate API key, store in Cloudflare environment variables, restart staging, verify on integrations dashboard.

Provider	Plan to start	Notes
Anthropic Claude	Build tier	Enable prompt caching for system prompts
MSG91	Volume	Submit DLT templates for OTP + transactional
Resend	Pro	Verify domain DKIM + SPF; warm IP for 14 days
PhonePe	Standard merchant	Complete onboarding (KYC, agreement); use sandbox first
100ms	Pay-as-you-go	Region: India South; create one room template
Pusher Channels	Startup	Cluster: ap2 (Mumbai)
Inngest	Hobby/Team	Create separate event keys for staging and prod

Provider	Plan to start	Notes
Better Stack	Team	Configure status page on status.ungghost.com

7.3 DNS layout

Host	Points to	Purpose
ungghost.com (apex)	301 → www.ungghost.com	Marketing redirect
www.ungghost.com	Cloudflare Container (production)	Production app
staging.ungghost.com	Cloudflare Container (staging)	Internal QA
preview-*.ungghost.com (wildcard)	Cloudflare Container (preview)	PR previews
status.ungghost.com	Better Stack	Public status page
docs.ungghost.com	Cloudflare Pages (static)	Public documentation
api.ungghost.com	Reserved	Future, if option B is adopted

All hosts use Cloudflare Universal SSL with TLS 1.3 minimum and HSTS preload (after 90-day verification).

7.4 Environment variable matrix

The application reads a single "env profile" at startup. Cloudflare Containers manages secrets per environment. The full surface:

Variable	Local	Preview	Staging	Production
NODE_ENV	development	production	production	production
MONGODB_URI	Docker Mongo	Atlas staging w/ PR prefix	Atlas staging	Atlas prod
NEXTAUTH_SECRET	dev secret	preview secret	staging secret	prod secret
NEXT_PUBLIC_APP_URL	http://localhost:3000	preview-{n}.ungghost.com	staging.ungghost.com	www.ungghost.com
UPSTASH_REDIS_REST_URL	—	shared staging Redis	shared staging Redis	prod Redis
UPSTASH_REDIS_REST_TOKEN	—	set	set	set
R2_*	—	preview bucket	staging bucket	prod bucket
SENTRY_DSN_SERVER	—	set	set	set
SENTRY_DSN_BROWSER	—	set	set	set

Variable	Local	Preview	Staging	Production
AXIOM_TOKEN	—	set	set	set
ANTHROPIC_API_KEY	(blank to mock)	sandbox	sandbox	live
MSG91_*	(blank to mock)	sandbox	sandbox	live
RESEND_API_KEY	(blank to mock)	test domain	test domain	live
PHONEPE_*	(blank to mock)	sandbox	sandbox	live
PUSHER_*	(blank to mock)	staging app	staging app	prod app
INNGEST_EVENT_KEY	(blank to mock)	preview env	staging env	prod env
100MS_*	(blank to mock)	dev	dev	live
GOOGLE_CLIENT_ID/SECRET	(blank)	test app	test app	live
LINKEDIN_CLIENT_ID/SECRET	(blank)	test app	test app	live
CRON_SECRET	dev	preview	staging	prod (rotated quarterly)

7.5 Dockerfile

The application packages as a standard Next.js standalone build. Below is the production Dockerfile, written for multi-stage cache reuse so subsequent builds are fast in CI.

```
# syntax=docker/dockerfile:1.7
FROM node:20-alpine AS deps
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci --omit=optional

FROM node:20-alpine AS build
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY . .
ENV NEXT_TELEMETRY_DISABLED=1
RUN npm run build # next build, output: 'standalone'

FROM node:20-alpine AS runtime
WORKDIR /app
ENV NODE_ENV=production NEXT_TELEMETRY_DISABLED=1 PORT=3000
RUN addgroup -g 1001 nodejs \
  && adduser -S nextjs -u 1001 -G nodejs
COPY --from=build --chown=nextjs:nodejs \
  /app/.next/standalone ./
COPY --from=build --chown=nextjs:nodejs \
  /app/.next/static ./next/static
COPY --from=build --chown=nextjs:nodejs \
  /app/public ./public
USER nextjs
EXPOSE 3000
HEALTHCHECK --interval=15s --timeout=3s \
  --start-period=20s --retries=3 \
  CMD wget -qO- http://127.0.0.1:3000/api/health || exit 1
CMD ["node", "server.js"]
```

7.6 Health endpoint

`/api/health` returns 200 only when Mongo and Redis are both reachable and the app version matches the deployed image. This endpoint is read by Cloudflare Containers for liveness, by GitHub Actions for post-deploy smoke, and by Better Stack for external uptime checks.

7.7 Region selection rationale

Mumbai (Atlas *ap-south-1*, Cloudflare Mumbai PoP, Pusher *ap2*, MSG91 India) is chosen for three reasons: (1) DPDP Act prefers in-country PII storage, (2) the target market is India-first, (3) PhonePe processes faster against India-resident merchants. The application code is region-agnostic; switching to another region in the future is a configuration change.

7.8 Secret rotation policy

- Database passwords: rotate every 90 days. Atlas supports rolling rotation without downtime.
- NEXTAUTH_SECRET: rotate every 180 days. Forces session invalidation — schedule a maintenance window.
- Third-party API keys: rotate when an engineer with access leaves the company, immediately.
- All rotation events logged in *docs/runbooks/rotation-log.md*.

8. CI/CD pipeline and deployment guide

CI/CD is the most leveraged engineering investment after testing. A good pipeline catches bugs in minutes, deploys safely without human intervention, and rolls back automatically when a deploy is bad. This section specifies the GitHub Actions workflow, environment matrix, and deployment procedures.

8.1 Branch and environment strategy

Trunk-based development with short-lived feature branches. Every PR opens a preview environment. Merging to *main* deploys to staging. Tagging a release (Git tag *v*.*.**) deploys to production. There are no long-lived develop / release branches.

Action	Trigger	Deploys to
Open PR / push to PR branch	GitHub PR event	Preview env (preview-{pr-num}.unghost.com)
Merge to main	Push to main	Staging (staging.unghost.com)
Create tag <i>v*.*.*</i>	Tag push	Production (www.unghost.com) — with approval gate
Manual rollback	GitHub Actions workflow_dispatch	Re-deploys previous image SHA

8.2 Pipeline stages

Every PR runs the full pipeline. Stages are ordered fastest-failing-first so feedback is quick:

- 1 **Lint** — ESLint, including boundaries rules.
- 2 **Typecheck** — `tsc --noEmit`.
- 3 **Unit tests** — Vitest, parallel, with coverage.
- 4 **Bundle-size check** — `size-limit`.
- 5 **Build** — `next build --output=standalone`.
- 6 **Integration tests** — API-level tests using an ephemeral `mongodb-memory-server`.
- 7 **End-to-end tests** — Playwright against a freshly built Docker image, with seeded data.
- 8 **Visual regression** — Chromatic diff for Storybook stories.
- 9 **Deploy to preview** — Build and push Docker image; deploy to Cloudflare Containers preview slot.
- 10 **Post-deploy smoke** — `curl` health endpoints on the live preview URL.
- 11 **Comment on PR** — Post preview URL into the PR description.

8.3 GitHub Actions workflow — main file

```
# .github/workflows/ci.yml
name: ci
on:
  pull_request:
  push:
    branches: [main]
    tags: ['v*']

concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress:
    ${{ github.event_name == 'pull_request' }}

jobs:
  verify:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20, cache: npm }
      - run: npm ci
      - run: npm run lint
      - run: npm run typecheck
      - run: npm run test:unit -- --coverage
      - uses: codecov/codecov-action@v4
      - run: npm run size
      - run: npm run build
      - run: npm run test:integration

  e2e:
    needs: verify
    runs-on: ubuntu-latest
    services:
      mongo:
        image: mongo:7
        ports: [27017:27017]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20, cache: npm }
      - run: npm ci
      - run: |
          npx playwright install --with-deps chromium
      - run: npm run seed:test
      - run: npm run build
      - run: npm run test:e2e
      - if: failure()
        uses: actions/upload-artifact@v4
        with:
          name: playwright-report
          path: playwright-report

  deploy-preview:
    if: github.event_name == 'pull_request'
    needs: [verify, e2e]
    runs-on: ubuntu-latest
    permissions:
      pull-requests: write
      contents: read
    env:
      PR: ${{ github.event.number }}
      IMG: registry.cloudflare.com/unghost-app
    steps:
      - uses: actions/checkout@v4
      - uses: docker/setup-buildx-action@v3
      - run: |
          docker build -t $IMG:pr-$PR .
          docker push $IMG:pr-$PR
    env:
```

```

    CLOUDFLARE_API_TOKEN:
      ${{ secrets.CLOUDFLARE_API_TOKEN }}
- run: |
  npx wrangler containers deploy \
    --name=unghost-preview-$PR \
    --image=$IMG:pr-$PR
- uses: actions/github-script@v7
  with:
    script: |
      const url =
        `https://preview-${context.issue.number}` +
        `.unghost.com`;
      github.rest.issues.createComment({
        ...context.repo,
        issue_number: context.issue.number,
        body: `Preview ready: ${url}`,
      });

deploy-staging:
  if: github.event_name == 'push'
    && github.ref == 'refs/heads/main'
  needs: [verify, e2e]
  runs-on: ubuntu-latest
  environment: staging
  env:
    IMG: registry.cloudflare.com/unghost-app
  steps:
    - uses: actions/checkout@v4
    - run: npm ci && npm run migrate:up
    env:
      MONGODB_URI:
        ${{ secrets.STAGING_MONGODB_URI }}
    - run: docker build -t $IMG:${{ github.sha }} .
    - run: docker push $IMG:${{ github.sha }}
    - run: |
      npx wrangler containers deploy \
        --name=unghost-staging \
        --image=$IMG:${{ github.sha }}
    - run: ./scripts/smoke.sh https://staging.unghost.com
    - if: failure()
      run: |
        npx wrangler containers rollback unghost-staging

deploy-production:
  if: startsWith(github.ref, 'refs/tags/v')
  needs: [verify, e2e]
  runs-on: ubuntu-latest
  environment: production # requires manual approval
  env:
    IMG: registry.cloudflare.com/unghost-app
    TAG: ${{ github.ref_name }}
  steps:
    - uses: actions/checkout@v4
    - run: npm ci && npm run migrate:up
    env:
      MONGODB_URI:
        ${{ secrets.PROD_MONGODB_URI }}
    - run: docker build -t $IMG:$TAG .
    - run: docker push $IMG:$TAG
    - run: |
      npx wrangler containers deploy \
        --name=unghost-prod \
        --image=$IMG:$TAG \
        --strategy=blue-green
    - run: ./scripts/smoke.sh https://www.unghost.com
    - if: failure()
      run: npx wrangler containers rollback unghost-prod
    - name: Notify Sentry release
      run: |
        npx sentry-cli releases new $TAG
        npx sentry-cli releases finalize $TAG

```

8.4 Database migrations

Migrations use *migrate-mongo* and are part of the deploy job, not a manual step.

- Folder: *server/db/migrations/*.
- File name: *YYYYMMDD_HHmm_description.js*.
- Every migration has both *up* and *down*. The *down* must be tested in staging.
- Migrations run before the new image is promoted to receive traffic. Failure blocks the deploy.
- Long-running migrations (>30 seconds) use a feature flag — deploy the code first behind a flag, run the migration out-of-band, flip the flag.

8.5 Rollback playbook

Rollbacks are a one-command operation; the runbook lives in *docs/runbooks/rollback.md*.

```
# Roll the production app back to the previous image SHA
npx wrangler containers rollback unghost-prod

# Or pin to a specific known-good SHA
npx wrangler containers deploy --name=unghost-prod --image=...:v1.4.2

# Roll a migration back (rarely – destructive)
MONGODB_URI=$PROD_URI npm run migrate:down -- --count=1
```

8.6 Smoke-test script

```
#!/usr/bin/env bash
# scripts/smoke.sh – runs after every deploy
set -euo pipefail
BASE=$1
echo "Smoke testing $BASE"

# 1. health endpoint
curl -fsS "$BASE/api/health" | jq -e '.ok == true'

# 2. login page renders
curl -fsS "$BASE/login" | grep -q 'Welcome back'

# 3. public API responds
curl -fsS "$BASE/api/bootcamps" | jq -e '. | length > 0'

# 4. version header matches deployed SHA
test "$(curl -sI "$BASE" | grep -i x-app-version | tr -d '\r' | awk '{print $2}')" = "$GITHUB_SHA"

echo "Smoke OK"
```

8.7 Secrets management

- **GitHub Secrets** — pipeline-time secrets (registry creds, Cloudflare API token, Sentry DSN for sourcemaps, Atlas migrator URI).
- **Cloudflare environment variables** — runtime app secrets, per environment.
- **1Password** — canonical home for every secret with sharing controls; engineers pull into local *.env* via *op* CLI.
- **Never commit** any *.env** file. *.gitignore* covers them; CI rejects PRs that contain new env-like files.

8.8 Approval gates

Production deploys require two human approvals via GitHub Environments protection rules. Approvers are configured in the *production* environment in repo settings. Staging deploys have no approval requirement to keep iteration fast.

8.9 Observability around deploys

- Every Sentry release is tagged with the Git SHA; errors auto-link to commits.
- Axiom logs include *app.version*, *app.environment*, *app.region* fields on every line.
- Better Stack receives a heartbeat after every successful deploy. Missed heartbeat means the deploy script crashed mid-flight.
- Slack #deploys gets one message per environment per deploy with the SHA, author, and link to the run.

9. 8-week hardening roadmap

The plan below converts everything in sections 1–8 into a week-by-week execution sequence. Each week has acceptance criteria; a week is "done" only when every criterion passes. The plan assumes two full-time engineers and access to ops support for provider setup.

Week 1 — Foundations: structure, design, contributing

Goal: every new contributor can clone, run, and find their way around in an hour.

- Folder restructure to *app/ · components/ · server/ · shared/* per section 5.
- Add ESLint *boundaries* rules and a CONTRIBUTING.md.
- Apply the new design guide tokens (assuming it lands this week).
- Set up Storybook with a story per atomic component.
- Open initial 5 ADRs: monolith, Cloudflare Containers, Atlas Mumbai, Redis cache, design swap.

Acceptance: a new engineer can clone, run `npm run dev`, and ship a hello-world PR within 90 minutes.

Week 2 — Security and persistence

Goal: no in-memory state, no plaintext passwords, no unbounded inputs.

- Bcrypt password path + secure cookie config (`httpOnly`, `secure`, `sameSite=lax`) wired into NextAuth.
- Persist OTP and email reset tokens in Redis with TTL.
- Persist support tickets and email templates to MongoDB collections.
- Add zod validators to every *app/api/**/**.ts route; reject unknown fields.
- Add CSRF protection on state-changing routes via NextAuth's built-in token where applicable.

Acceptance: zod schema present on 100 percent of POST/PUT/PATCH/DELETE routes; e2e auth flow uses bcrypt-hashed seed users.

Week 3 — Test suite

Goal: every critical user flow is covered by an automated test.

- Vitest unit suite for every *server/{domain}/service.ts*.
- Playwright e2e flows: signup, login, apply to mission, stage advance, bootcamp enrol, AI Coach chat, recruiter post-job, admin moderation.
- Mongoose tests use *mongodb-memory-server* for isolation.
- Coverage gate in CI: fail the build below 70 percent statement coverage on changed files.

Acceptance: green CI on a 30-PR rolling window; flake rate below 2 percent.

Week 4 — Observability and limits

Goal: when something breaks, we know within 60 seconds.

- Sentry SDK in server and browser; release wiring tied to CI.
- Pino structured logs with correlation-id middleware on every request.

- Axiom transport for logs; saved queries created.
- Upstash Redis rate-limit middleware on `/api/coach`, `/api/otp`, `/api/email/*`, `/api/auth/*`.
- R2 file storage with presigned URLs for resume PDF uploads.

Acceptance: a synthetic 5xx in staging shows up in Sentry + Axiom + Slack #engineering within 60 seconds.

Week 5 — Database and caching

Goal: queries are fast at 10x today's data and cache hit-rate is measurable.

- Audit every read path and add compound indexes as needed; document in `server/db/indexes.ts`.
- Run an EXPLAIN audit script in CI on every PR that touches a model.
- Add a read-through cache wrapper around the four highest-traffic queries (bootcamp catalogue, public missions, company profile, dashboard counts).
- Wire migrate-mongo with a `migrate:up` script.
- Atlas slow-query alerts to PagerDuty at >500 ms.

Acceptance: p95 of any indexed query ≤ 80 ms at 10x current data volume; cache hit-rate ≥ 80 percent on the four wrapped queries.

Week 6 — Cloudflare and Docker

Goal: staging.unghost.com is live, healthy, and reachable from the public internet.

- Productionised Dockerfile per section 7.5.
- Cloudflare account, Containers project, DNS, R2 buckets all provisioned per section 7.
- MongoDB Atlas M30 in Mumbai, Upstash Redis, Sentry, Axiom, Better Stack all online.
- Staging environment fully deployed with sandbox provider keys.
- Health endpoint and smoke script working end-to-end.

Acceptance: staging.unghost.com serves the application using sandbox keys; uptime ≥ 99.5 percent for 5 consecutive days.

Week 7 — CI/CD and PR previews

Goal: every PR gets a live URL; staging auto-deploys; rollback is one click.

- GitHub Actions workflow per section 8.3, including preview, staging, and production jobs.
- Approval gate configured on the production environment.
- Sentry release tracking wired into prod deploys.
- Smoke + auto-rollback in production deploy job.
- Documentation: `rollback.md`, `incident-response.md`, `on-call.md` runbooks.

Acceptance: a contrived bad deploy auto-rolls-back within 90 seconds and reaches Slack #prod with a link to the run.

Week 8 — Real keys, load, launch readiness

Goal: live providers, 10k-user load test passes, go/no-go meeting held.

- Swap sandbox keys for live Anthropic, MSG91, Resend, PhonePe, Pusher, 100ms, Inngest.

- Real 100ms SDK integrated in the live room.
- Load test with k6 simulating 10,000 concurrent sessions; results documented; capacity tuned.
- DPDP data-export and erasure endpoints implemented.
- Go-live runbook: pre-cutover, cutover, post-cutover checklists.
- Production deploy at the end of the week — soft launch to invited beta users.

Acceptance: 10k-user k6 test holds error rate < 0.1 percent and p95 <= 800 ms; legal sign-off on DPDP; production live.

Risk register

Risk	Likelihood	Impact	Mitigation
Provider onboarding (MSG91 DLT, PhonePe KYC) slower than expected	High	Schedule slip	Start Week 1 in parallel; sandbox covers dev
Design guide late or vague	Medium	Week 1 slip	Apply guide token-by-token; UI logic untouched
Test backfill slower than 10 days	Medium	Weak coverage	Prioritise critical paths; ship at 50 percent then 70 percent
Atlas/Cloudflare regional issue	Low	Outage	Better Stack alerts; Atlas cross-region read replica when scale warrants
Mongoose performance surprise at 10k load	Medium	Scaling rework	Index audit in Week 5; replicas + cache in Week 5
Live 100ms minutes cost spike	Medium	Cost	Per-room time cap + monthly budget alert

Appendix

A1. Service inventory

Service	Purpose	Owner
Cloudflare (CDN/WAF/Containers/R2/DNS)	Edge, runtime, object storage	DevOps
MongoDB Atlas (Mumbai, M30)	Primary OLTP database	Backend
Upstash Redis (Mumbai)	Cache, rate-limit, sessions, OTP store	Backend
Sentry	Error tracking + performance traces	Eng leadership
Axiom	Structured logs	Eng leadership
Better Stack	External uptime + status page	Eng leadership
Anthropic Claude	LLM provider	Product
MSG91	SMS + OTP delivery	Product
Resend	Transactional email	Product
PhonePe	Payment gateway	Finance
100ms	Video / live sessions	Product
Pusher Channels	Realtime messaging	Product
Inngest	Background jobs	Backend
GitHub + Actions	Source control + CI/CD	DevOps
1Password	Secret storage	All engineers

A2. Cost roll-up (10k MAU)

Approximate steady-state monthly cost in USD, excluding payment-processor variable fees:

Bucket	USD / month
Infrastructure (CF, Atlas, Redis, R2)	~ 575
Observability (Sentry, Axiom, Better Stack)	~ 180
Product providers (Claude, MSG91, Resend, 100ms, Pusher, Inngest)	~ 650
Total fixed	~ 1,405 USD
Variable (PhonePe transaction fees)	~ 1.99% + GST per txn

A3. Glossary

Term	Meaning
DPDP	Digital Personal Data Protection Act 2023 — India's GDPR-equivalent. Drives data residency + erasure rights.
SLA	Service-Level Agreement. unGhost's anti-ghost guarantee: recruiter responds within a stated number of hours or student gets a refund.
RSC	React Server Components — Next.js App Router feature; renders components on the server.
MAU	Monthly active users.
RUM	Real-User Monitoring — performance data collected from real user browsers.
WAF	Web Application Firewall.
PITR	Point-In-Time Recovery — database backup feature.
ADR	Architecture Decision Record.
PR preview	An ephemeral environment built for an in-progress pull request.

A4. Open questions for tech-head review

- Is Cloudflare Containers the right runtime, or would the team prefer Fly.io / Railway / Render given familiarity?
- Do we add a separate read replica from week 1, or wait for the load test in week 8 to confirm need?
- Should we co-locate the team for the first two weeks of the hardening sprint?
- What is the launch GTM plan, and does it suggest a different DAU/peak ratio than the 20%/500 assumed here?
- Are there compliance requirements beyond DPDP (e.g. SOC 2 for enterprise recruiter contracts) we should plan for now rather than later?

End of document.